FlashAttention-2 论文精读

Faster Attention with Better Parallelism and Work Partitioning

作者: Tri Dao (Princeton University / Stanford University)

Contents

1 摘要 (Abstract) **2**

2 引言 (Introduction) **4**

3 背景 (Background) **6**
3.1 硬件特性 (Hardware Characteristics) 6
3.2 标准注意力实现 (Standard Attention Implementation) 7
3.3 FlashAttention (FlashAttention) 8
3.3.1 前向传播 9
3.3.2 反向传播 11

4 FlashAttention-2：算法、并行性与工作分配 **12**
4.1 算法 (Algorithm) 12
4.1.1 前向传播 12
4.1.2 反向传播 16
4.2 并行性 (Parallelism) 17
4.3 Warp 间的工作分配 (Work Partitioning Between Warps) 19

5 实验验证 (Empirical Validation) **21**
5.1 注意力基准测试 (Benchmarking Attention) 21
5.2 端到端性能 (End-to-end Performance) 25

6 讨论与未来方向 (Discussion and Future Directions) **27**

附录：参考文献一览表 **28**

1 摘要 (Abstract)

原文翻译

将 Transformer 扩展到更长的序列长度是过去几年的一个主要问题，这有望提升语言建模和高分辨率图像理解的性能，并解锁代码、音频和视频生成等新应用。注意力层是扩展到更长序列的主要瓶颈，因为其运行时间和内存需求随序列长度呈二次增长。
FlashAttention [Dao et al., 2022] 【FlashAttention 原始论文：提出利用 GPU 非对称存储层次结构来加速注意力计算的方法，是本文的直接前作】 利用 GPU 非对称存储层次结构实现了显著的内存节省（线性而非二次）和运行时加速（相比优化的基线快 2–4 倍），且没有近似。然而，FlashAttention 仍远不如优化的矩阵乘法（GEMM）操作快，仅达到理论最大 FLOPs/s 的 25–40%。
我们观察到，这种低效率是由于 GPU 上不同线程块和 warp 之间的工作分配不优导致的，造成了低占用率或不必要的共享内存读写。
我们提出 FlashAttention-2，通过更好的工作分配来解决这些问题。具体而言，我们（1）调整算法以减少非矩阵乘法 FLOPs 的数量，（2）将注意力计算在不同线程块间并行化，即使对于单个注意力头也是如此，以提高占用率，（3）在每个线程块内，在 warp 之间分配工作以减少通过共享内存的通信。
这些改进带来了约 2 倍于 FlashAttention 的加速，在 A100 上达到理论最大 FLOPs/s 的 50–73%，接近 GEMM 操作的效率。我们通过端到端训练 GPT 风格模型实证验证，FlashAttention-2 在每个 A100 GPU 上达到高达 225 TFLOPs/s 的训练速度（72% 模型 FLOPs 利用率）。[a]

[a] FlashAttention-2 可在 https://github.com/Dao-AILab/flash-attention 获取

通俗解释

这段在说什么?
想象你在读一本很长的书，每读到一个词都需要回看前面**所有**词来理解上下文——这就是 Transformer 中“注意力”机制在做的事。书越长（序列越长），需要回看的量就呈**平方级**增长：序列长度翻倍，计算量变成 4 倍。
FlashAttention（第一代 FlashAttention）就像发明了一种聪明的”快速翻书法”：不是一次性把整本书摊开（占大量桌面/显存），而是一次只翻几页来处理，大幅减少了内存占用和运行时间。
但 FlashAttention 还有一个问题：GPU 上”工人们”（线程）的分工不太合理，就像一个工厂里有些工人在闲着，有些工人在做重复搬运。FlashAttention-2 通过三招优化了这个分工问题：

1. 减少非矩阵乘法的杂活（GPU 做矩阵乘法特别快，做其他运算就慢得多）
2. 让更多工人同时干活（提高并行度/占用率）
3. 工人之间少传话（减少共享内存通信）

最终结果：比 FlashAttention 再快约 2 倍，达到 GPU 理论性能的 50–73%，非常接近最快的矩阵乘法运算了。

关键概念

- FlashAttention-2 是 FlashAttention 的改进版，核心目标是**缩小注意力计算与 GEMM 之间的性能差距**
- 三大改进：减少非 matmul FLOPs、序列维度并行化、优化 warp 间工作分配

- 在 A100 上达到理论峰值的 50–73%，端到端训练达到 225 TFLOPs/s

2 引言 (Introduction)

原文翻译

将 Transformer [Vaswani et al., 2017] 【Attention is All You Need：提出 Transformer 架构的开创性论文，引入自注意力机制，是现代大语言模型的基础架构】 扩展到更长的上下文长度是一个挑战，因为其核心的注意力层的运行时间和内存需求随输入序列长度呈二次增长。理想情况下，我们希望突破标准的 2k 序列长度限制，训练模型来理解书籍、高分辨率图像和长视频。仅在过去一年中，就出现了几个上下文长度远超以往的语言模型：GPT-4 [OpenAI, 2023] 【OpenAI 发布的 GPT-4 技术报告，上下文长度达 32k，证明了长上下文的工程可行性】 上下文长度 32k，MosaicML 的 MPT 上下文长度 65k，以及 Anthropic 的 Claude 上下文长度 100k。长文档查询和故事写作等新兴应用已证明了对这种长上下文模型的需求。

为了降低注意力在这种长上下文上的计算需求，已有大量方法被提出来近似注意力 [Kitaev et al., 2020] 【Reformer：使用局部敏感哈希近似注意力的方法】、[Roy et al., 2021] 【Routing Transformers：使用路由机制的稀疏注意力】、[Wang et al., 2020] 【Linformer：将注意力矩阵低秩近似以降低到线性复杂度】、[Katharopoulos et al., 2020] 【Linear Attention：将 Transformer 视为 RNN，用线性注意力加速】、[Choromanski et al., 2020] 【Performer：使用随机特征近似 softmax 注意力核】、[Beltagy et al., 2020] 【Longformer：使用局部滑动窗口+全局注意力处理长文档】、[Zaheer et al., 2020] 【BigBird：结合局部、全局和随机注意力模式】、[Chen et al., 2021] 【ScatterBrain：统一稀疏和低秩注意力近似方法】。虽然这些方法有一些应用场景，但据我们所知，大多数大规模训练仍然使用标准注意力。

受此启发，Dao et al. [2022] 【FlashAttention 原始论文：本文的直接前作】 提出重新排列注意力计算顺序，并利用经典技术（分块、重计算）来显著加速注意力并将内存使用从序列长度的二次降低到线性。这带来了 2–4 倍于优化基线的实际加速，高达 10–20 倍的内存节省，且没有近似，因此 FlashAttention 在 Transformer 的大规模训练和推理中得到了广泛采用。

然而，随着上下文长度进一步增加，FlashAttention 仍远不如矩阵乘法（GEMM）等其他原语高效。具体而言，虽然 FlashAttention 已经比标准注意力实现快 2–4 倍，但前向传播仅达到设备理论最大 FLOPs/s 的 30–50%（见图 5），而反向传播更具挑战性，在 A100 GPU 上仅达到最大吞吐量的 25–35%（见图 6）。相比之下，优化的 GEMM 可达到理论最大设备吞吐量的 80–90%。通过仔细的性能分析（profiling），我们观察到 FlashAttention 在 GPU 上不同线程块和 warp 之间的工作分配仍不理想，导致低占用率或不必要的共享内存读写。

在 FlashAttention 的基础上，我们提出 FlashAttention-2，通过更好的并行性和工作分配来解决这些挑战。

1. 在第 4.1 节中，我们调整算法以减少非矩阵乘法 FLOPs 的数量，同时不改变输出。虽然非矩阵乘法 FLOPs 仅占总 FLOPs 的一小部分，但它们执行时间更长，因为 GPU 拥有矩阵乘法的专用单元，因此矩阵乘法吞吐量可比非矩阵乘法吞吐量高 16 倍。因此，减少非矩阵乘法 FLOPs 并尽可能多地花时间在矩阵乘法 FLOPs 上是至关重要的。

2. 我们提出在序列长度维度上同时并行化前向传播和反向传播，作为对 batch 和头数维度并行化的补充。这在序列较长（因此 batch size 通常较小）的情况下提高了占用率（GPU 资源的利用率）。

3. 即使在一个注意力计算块内，我们也在线程块的不同 warp 之间分配工作，以减少通信和共享内存读写。

在第 5 节中，我们实证验证了 FlashAttention-2 相比 FlashAttention 带来了显著加速。不同设置（有无因果掩码、不同头维度）的基准测试表明，FlashAttention-2 比

FlashAttention 快约 2 倍，前向传播达到理论最大吞吐量的 73%，反向传播达到 63%。当端到端用于训练 GPT 风格模型时，我们达到了每 A100 GPU 高达 225 TFLOPs/s 的训练速度。

通俗解释

引言部分讲了三件事：
1. 问题在哪？ Transformer 的注意力是 $O(N^2)$ 的，序列越长越慢。虽然有很多”近似注意力”方法，但实际大规模训练几乎都还在用标准注意力。
2. FlashAttention 做了什么？ 通过分块（tiling）+ 重计算（recomputation）的经典技巧，不存储中间的 $N \times N$ 矩阵，将内存从 $O(N^2)$ 降到 $O(N)$，速度快了 2–4 倍。
3. 为什么还需要 FlashAttention-2？ FlashAttention 虽然比标准实现快，但跟 GPU 上最快的矩阵乘法（GEMM）比，还差很远——前向只达到峰值的 30–50%，反向更差。罪魁祸首是 GPU 上的”工作分配”（work partitioning）做得不好。FlashAttention-2 通过三个优化把这个差距大幅缩小。
关键数字： GEMM 能到 80–90% 峰值，FlashAttention 只有 25–50%，FlashAttention-2 达到 50–73%。

注意事项

matmul vs non-matmul 的性能差异是理解本文的关键。在 A100 上：

- 矩阵乘法（matmul）：312 TFLOPs/s（FP16/BF16，使用 Tensor Core）
- 非矩阵乘法（标量运算等）：19.5 TFLOPs/s（FP32）
- 差距：$312/19.5 = 16\times$！

所以即使非 matmul 运算在 FLOP 数量上很少，它们在时间上的占比可能很大。这就是为什么减少非 matmul FLOPs 如此重要。

3 背景 (Background)

原文翻译

我们提供一些关于 GPU 性能特征和执行模型的背景知识。我们还描述注意力的标准实现以及 FlashAttention。

3.1 硬件特性 (Hardware Characteristics)

原文翻译

GPU 性能特性。 GPU 由计算元件（如浮点运算单元）和存储层次结构组成。大多数现代 GPU 包含专门加速低精度矩阵乘法的单元（如 Nvidia GPU 上用于 FP16/BF16 矩阵乘法的 Tensor Core）。存储层次结构包括高带宽内存（HBM）和片上 SRAM（又称共享内存）。以 A100 GPU 为例，它有 40–80GB 的 HBM，带宽为 1.5–2.0 TB/s，以及每个 108 个流式多处理器（SM）上 192KB 的片上 SRAM，带宽估计约为 19 TB/s [Jia et al., 2018]【Volta GPU 微基准测试：测量 GPU 内存层次的带宽和延迟】、[Jia & Van Sandt, 2021]【Ampere GPU 微基准测试：测量 A100 的内存性能参数】。由于 L2 缓存不可由程序员直接控制，本文讨论聚焦于 HBM 和 SRAM。
执行模型。 GPU 拥有大量线程来执行一个操作（称为 kernel）。线程被组织成线程块（thread block），线程块被调度在流式多处理器（SM）上运行。在每个线程块内，线程被分组为 warp（一组 32 个线程）。warp 内的线程可以通过快速的 shuffle 指令通信或协作执行矩阵乘法。线程块内的 warp 可以通过读写共享内存来通信。每个 kernel 从 HBM 加载输入到寄存器和 SRAM，进行计算，然后将输出写回 HBM。

通俗解释

把 GPU 想象成一个巨大的工厂：
存储层次就像工厂的仓库系统：

- **HBM（高带宽内存）** = 大仓库（40–80GB），搬运速度约 2 TB/s。原料和成品都放这里。
- **SRAM（共享内存）** = 工位旁的小桌子（每个车间 192KB），搬运速度约 19 TB/s（快 10 倍！）。但空间非常小。

线程组织就像工人的编制：

- **线程**= 单个工人
- **Warp**= 一个小组（32 人），组内可以直接传话（shuffle 指令）
- **线程块**= 一个车间，车间内小组之间通过公告栏（共享内存）通信
- **SM**= 车间的物理空间（A100 有 108 个）

关键瓶颈：GPU 计算很快，但数据从大仓库（HBM）搬到小桌子（SRAM）很慢。这就是为什么要尽量减少 HBM 访问。

关键概念

A100 GPU 关键数字:

- HBM: 40–80 GB, 带宽 1.5–2.0 TB/s
- SRAM: 每个 SM 192 KB, 带宽 ~19 TB/s
- SM 数量: 108
- Tensor Core matmul: 312 TFLOPs/s (FP16/BF16)
- 非 matmul: 19.5 TFLOPs/s (FP32)

3.2 标准注意力实现 (Standard Attention Implementation)

原文翻译

给定输入序列 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$，其中 N 是序列长度，d 是头维度，我们要计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}, \tag{1}$$

其中 softmax 按行应用。[a] 对于多头注意力（MHA），这个计算在多个头上并行执行，并在 batch 维度上并行。
注意力的反向传播如下进行。设 $\mathbf{dO} \in \mathbb{R}^{N \times d}$ 为损失函数对 $\mathbf{O}$ 的梯度。通过链式法则（即反向传播）:

$$\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d} \tag{2}$$
$$\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N} \tag{3}$$
$$\mathbf{dS} = \mathrm{dsoftmax}(\mathbf{dP}) \in \mathbb{R}^{N \times N} \tag{4}$$
$$\mathbf{dQ} = \mathbf{dS}\mathbf{K} \in \mathbb{R}^{N \times d} \tag{5}$$
$$\mathbf{dK} = \mathbf{Q}\mathbf{dS}^\top \in \mathbb{R}^{N \times d}, \tag{6}$$

其中 dsoftmax 是按行应用的 softmax 梯度（反向传播）。可以推导出，如果 $p = \mathrm{softmax}(s)$（对于某些向量 s 和 p），则对于输出梯度 dp，输入梯度 $ds = (\mathrm{diag}(p) - pp^\top)dp$。
标准注意力实现将矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 物化到 HBM 中，这需要 $O(N^2)$ 内存。通常 $N \gg d$（典型的 N 在 1k–8k 之间，d 约为 64–128）。标准注意力实现 (1) 调用矩阵乘法（GEMM）子程序计算 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$，将结果写入 HBM，然后 (2) 从 HBM 加载 $\mathbf{S}$ 以计算 softmax 并将结果 $\mathbf{P}$ 写入 HBM，最后 (3) 调用 GEMM 得到 $\mathbf{O} = \mathbf{P}\mathbf{V}$。由于大部分操作受内存带宽限制，大量内存访问导致了较慢的实际运行时间。此外，由于需要物化 $\mathbf{S}$ 和 $\mathbf{P}$，所需内存为 $O(N^2)$。而且，反向传播需要保存 $\mathbf{P} \in \mathbb{R}^{N \times N}$ 以计算梯度。

[a] 为了表述清晰，我们省略了 $\mathbf{Q}\mathbf{K}^\top$ 的缩放（通常除以 $1/\sqrt{d}$），以及可选的对 $\mathbf{S}$ 的逐元素掩码和/或对 $\mathbf{P}$ 的 dropout。

注意力公式详细推导

前向传播

设 $N = 4$（4 个 token），$d = 2$（头维度 2），给定：

$$\mathbf{Q} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}, \quad \mathbf{K} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{V} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{pmatrix}$$

步骤 1：计算 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$
$\mathbf{Q}$（4×2）乘以 $\mathbf{K}^\top$（2×4）$= \mathbf{S}$（4×4）：

$$\mathbf{S} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

步骤 2：按行 softmax $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$
以第 1 行为例：$s = [1, 0, 1, 0]$，$e^s = [e, 1, e, 1]$，$\sum = 2e + 2 \approx 7.44$
$p_1 \approx [0.365, 0.134, 0.365, 0.134]$
步骤 3：$\mathbf{O} = \mathbf{P}\mathbf{V}$
$\mathbf{P}$（4×4）乘以 $\mathbf{V}$（4×2）$= \mathbf{O}$（4×2）。每行 $\mathbf{O}_i$ 是所有 $\mathbf{V}$ 行的加权平均，权重就是 softmax 后的注意力概率。
反向传播
关键公式是 softmax 的梯度：$ds = (\mathrm{diag}(p) - pp^\top)dp$。
展开来看：$ds_i = p_i \cdot dp_i - p_i \cdot \sum_k p_k \cdot dp_k = p_i(dp_i - \sum_k p_k \cdot dp_k)$
即：$\mathbf{dS} = \mathbf{P} \circ (\mathbf{dP} - D)$，其中 $D = \mathrm{rowsum}(\mathbf{P} \circ \mathbf{dP})$。
这里 $\circ$ 表示逐元素乘法（Hadamard 积），D 是一个列向量，每行存储该行 $\mathbf{P} \circ \mathbf{dP}$ 的和。

通俗解释

标准注意力为什么慢?
想象一个阅读理解过程：

1. 先算”谁对谁重要”（$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$），生成一张 $N \times N$ 的”关系表”
2. 把这张表做 softmax 变成概率（$\mathbf{P}$），又是一张 $N \times N$ 的表
3. 用这个概率表去加权取值（$\mathbf{O} = \mathbf{P}\mathbf{V}$）

问题在于，那张 $N \times N$ 的大表必须完整地写入 GPU 大仓库（HBM），然后再读出来。当 $N = 8192$ 时，这张表有 $8192^2 \approx 6700$ 万个元素，每个 2 字节（FP16），就是约 128 MB。反向传播还需要保存这张表，内存需求更大。

3.3 FlashAttention (FlashAttention)

原文翻译

为了在 GPU 等硬件加速器上加速注意力，Dao et al. [2022] 提出了一种算法，在保持相同输出（无近似）的同时减少内存读写。

3.3.1 前向传播

原文翻译

FlashAttention 应用经典的分块（tiling）技术来减少内存 IO：(1) 从 HBM 加载输入块到 SRAM，(2) 对该块计算注意力，然后 (3) 更新输出，而不将大的中间矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 写入 HBM。由于 softmax 耦合了整行或行块，在线 softmax [Milakov & Gimelshein, 2018]【Online Softmax：提出在线计算 softmax 归一化因子的方法，避免需要预先知道全行最大值】、[Rabe & Staats, 2021]【Self-attention does not need $O(n^2)$ memory：证明自注意力可以用 $O(n)$ 内存计算】可以将注意力计算拆分成块，对每个块的输出进行重缩放，最终获得正确结果（无近似）。通过显著减少内存读写量，FlashAttention 实现了 2–4 倍的加速。

我们描述在线 softmax 技术以及它如何在注意力中使用。为简单起见，考虑注意力矩阵 $\mathbf{S}$ 的一个行块，形如 $\begin{bmatrix}\mathbf{S}^{(1)} & \mathbf{S}^{(2)}\end{bmatrix}$，其中 $\mathbf{S}^{(1)}, \mathbf{S}^{(2)} \in \mathbb{R}^{B_r \times B_c}$，$B_r$ 和 B_c 分别为行和列块大小。我们要计算该行块的 softmax 并乘以 value，形如 $\begin{bmatrix}\mathbf{V}^{(1)} \\ \mathbf{V}^{(2)}\end{bmatrix}$，其中 $\mathbf{V}^{(1)}, \mathbf{V}^{(2)} \in \mathbb{R}^{B_c \times d}$。

标准 softmax 会计算：

$$
\begin{aligned}
m &= \max(\mathrm{rowmax}(\mathbf{S}^{(1)}), \mathrm{rowmax}(\mathbf{S}^{(2)})) \in \mathbb{R}^{B_r} \\
\ell &= \mathrm{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \mathrm{rowsum}(e^{\mathbf{S}^{(2)}-m}) \in \mathbb{R}^{B_r} \\
\mathbf{O} &= \mathrm{diag}(\ell)^{-1}\left(e^{\mathbf{S}^{(1)}-m}\mathbf{V}^{(1)} + e^{\mathbf{S}^{(2)}-m}\mathbf{V}^{(2)}\right) \in \mathbb{R}^{B_r \times d}.
\end{aligned}
$$

在线 softmax 则计算每个块的“局部” softmax 并在最后重缩放以获得正确输出：

$$
\begin{aligned}
m^{(1)} &= \mathrm{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \mathrm{rowsum}(e^{\mathbf{S}^{(1)}-m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{P}}^{(1)} &= \mathrm{diag}(\ell^{(1)})^{-1}e^{\mathbf{S}^{(1)}-m^{(1)}} \in \mathbb{R}^{B_r \times B_c} \\
\mathbf{O}^{(1)} &= \tilde{\mathbf{P}}^{(1)}\mathbf{V}^{(1)} = \mathrm{diag}(\ell^{(1)})^{-1}e^{\mathbf{S}^{(1)}-m^{(1)}}\mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \mathrm{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)}-m^{(2)}}\ell^{(1)} + \mathrm{rowsum}(e^{\mathbf{S}^{(2)}-m^{(2)}}) = \ell \\
\mathbf{O}^{(2)} &= \mathrm{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)}\mathbf{V}^{(2)} = \mathbf{O}.
\end{aligned}
$$

在线 Softmax 详细推导

核心问题：softmax 需要知道整行的最大值 m 才能计算 e^{s_i-m}，但我们一次只能看到一部分数据。

解决方案：边处理边更新统计量。

假设我们有两块 $\mathbf{S}^{(1)}$ 和 $\mathbf{S}^{(2)}$（为简单起见，考虑单行）：

处理第 1 块：

- 局部最大值 $m^{(1)} = \max(\mathbf{S}^{(1)})$
- 局部求和 $\ell^{(1)} = \sum e^{\mathbf{S}_i^{(1)}-m^{(1)}}$
- 局部输出 $\mathbf{O}^{(1)} = \frac{1}{\ell^{(1)}}\sum e^{\mathbf{S}_i^{(1)}-m^{(1)}}\mathbf{V}_i^{(1)}$

处理第 2 块时需要“纠正”：

新的全局最大值 $m^{(2)} = \max(m^{(1)}, \max(\mathbf{S}^{(2)}))$

旧的 $\ell^{(1)}$ 需要乘以修正因子 $e^{m^{(1)}-m^{(2)}}$（因为之前减的是 $m^{(1)}$，现在应该减 $m^{(2)}$）：

$$e^{s-m^{(1)}} = e^{s-m^{(2)}} \cdot e^{m^{(2)}-m^{(1)}}$$

所以 $e^{s-m^{(2)}} = e^{s-m^{(1)}} \cdot e^{m^{(1)}-m^{(2)}}$
同理，旧的 $\mathbf{O}^{(1)}$ 也需要乘以这个修正因子再归一化。
数值例子：设 $\mathbf{S}^{(1)} = [2, 1]$，$\mathbf{S}^{(2)} = [3, 0]$
第 1 块：$m^{(1)} = 2$，$\ell^{(1)} = e^0 + e^{-1} = 1 + 0.368 = 1.368$
第 2 块：$m^{(2)} = \max(2, 3) = 3$
修正 ℓ：$\ell^{(2)} = e^{2-3} \cdot 1.368 + e^{3-3} + e^{0-3} = 0.368 \cdot 1.368 + 1 + 0.050 = 0.503 + 1.050 = 1.553$
验证：直接算全行 $\ell = e^{2-3} + e^{1-3} + e^{3-3} + e^{0-3} = 0.368 + 0.135 + 1 + 0.050 = 1.553$ ✓

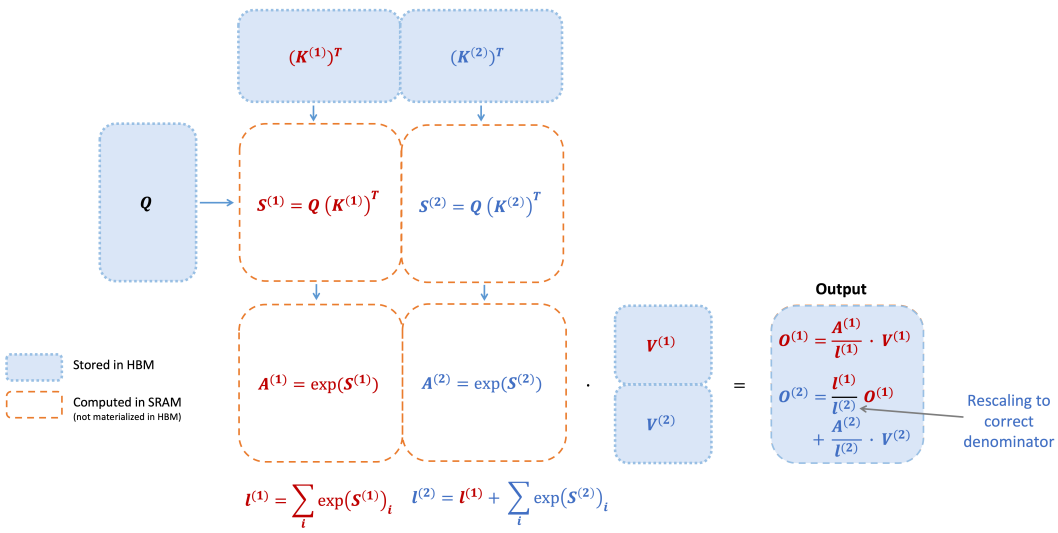


Figure 1: FlashAttention 前向传播示意图。当 key **K** 被分为两个块、value **V** 也被分为两个块时的计算流程。通过对每个块计算注意力并重缩放输出，我们在最后得到正确答案，同时避免了中间矩阵 **S** 和 **P** 的昂贵内存读写。图中简化了 softmax 中减去行最大值的步骤。

图 1 解读

这张图展示了 FlashAttention 的核心思想：
左侧展示了输入矩阵 **Q**, **K**, **V** 的分块方式。**K** 和 **V** 各被分成两块。
中间过程：

- 先加载第 1 块 $\mathbf{K}^{(1)}, \mathbf{V}^{(1)}$ 到 SRAM
- 计算局部注意力得分 $\mathbf{S}^{(1)} = \mathbf{Q} \cdot (\mathbf{K}^{(1)})^\top$
- 做局部 softmax，得到局部输出 $\mathbf{O}^{(1)}$
- 再加载第 2 块 $\mathbf{K}^{(2)}, \mathbf{V}^{(2)}$ 到 SRAM
- 计算 $\mathbf{S}^{(2)}$，更新全局统计量，修正之前的输出

核心优势：中间的 $N \times N$ 矩阵 **S** 和 **P** 从未完整地存在于 HBM 中！每次只有一小块在 SRAM 里。

3.3.2 反向传播

原文翻译

在反向传播中，通过在 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 的输入块已经加载到 SRAM 后重新计算注意力矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 的值，FlashAttention 避免了存储大的中间值。通过不需要保存大小为 $N \times N$ 的大矩阵 $\mathbf{S}$ 和 $\mathbf{P}$，FlashAttention 根据序列长度实现了 10–20 倍的内存节省（所需内存从序列长度 N 的二次变为线性）。反向传播同样由于减少了内存读写而实现了 2–4 倍的加速。反向传播对第 3.3 节中的公式应用分块。虽然反向传播在概念上比前向传播简单（没有 softmax 重缩放），但实现要复杂得多。这是因为在反向传播中需要在 SRAM 中保留更多值以执行 5 次矩阵乘法，而前向传播只有 2 次矩阵乘法。

通俗解释

反向传播的关键洞察：既然前向传播时我们没有保存 $\mathbf{S}$ 和 $\mathbf{P}$，那反向传播需要它们怎么办？答案是**重新计算**（recomputation）！
当 $\mathbf{Q}_i, \mathbf{K}_j, \mathbf{V}_j$ 已经加载到 SRAM 时，重新算一遍 $\mathbf{S}_{ij}$ 和 $\mathbf{P}_{ij}$ 的代价很小（计算快），远比从 HBM 读取一个巨大的 $N \times N$ 矩阵要快（IO 慢）。
这就是”用计算换内存”的经典策略——多算一次来避免多存一次。

4 FlashAttention-2：算法、并行性与工作分配

原文翻译

我们描述 FlashAttention-2 算法，其中包括对 FlashAttention 的几项调整以减少非矩阵乘法 FLOPs 的数量。然后描述如何在不同线程块间并行化计算以充分利用 GPU 资源。最后描述如何在一个线程块内的不同 warp 之间分配工作以减少共享内存访问量。这些改进带来了 2–3 倍的加速，在第 5 节中得到验证。

4.1 算法 (Algorithm)

原文翻译

我们调整 FlashAttention 的算法以减少非矩阵乘法 FLOPs 的数量。这是因为现代 GPU 拥有使矩阵乘法更快的专用计算单元（如 Nvidia GPU 上的 Tensor Core）。以 A100 GPU 为例，其 FP16/BF16 矩阵乘法的最大理论吞吐量为 312 TFLOPs/s，而非矩阵乘法 FP32 仅为 19.5 TFLOPs/s。换一种思考方式：每个非矩阵乘法 FLOP 的代价是矩阵乘法 FLOP 的 16 倍。为了保持高吞吐量（如超过最大理论 TFLOPs/s 的 50%），我们希望尽可能多地花时间在矩阵乘法 FLOPs 上。

4.1.1 前向传播

原文翻译

我们重新审视第 3.3 节中的在线 softmax 技巧，进行两项小调整以减少非矩阵乘法 FLOPs：

1. 我们不需要用 $\mathrm{diag}(\ell^{(2)})^{-1}$ 来重缩放输出更新的两项：

$$\mathbf{O}^{(2)} = \mathrm{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \mathrm{diag}(\ell^{(2)})^{-1}e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)}.$$

   我们可以改为维护一个“未缩放”版本的 $\mathbf{O}^{(2)}$ 并保留统计量 $\ell^{(2)}$：

$$\tilde{\mathbf{O}}^{(2)} = \mathrm{diag}(e^{m^{(1)}-m^{(2)}})\tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)}.$$

   仅在循环的最后才将最终的 $\tilde{\mathbf{O}}^{(\mathrm{last})}$ 乘以 $\mathrm{diag}(\ell^{(\mathrm{last})})^{-1}$ 以获得正确输出。

2. 反向传播不需要同时保存最大值 $m^{(j)}$ 和指数和 $\ell^{(j)}$。只需存储 logsumexp $L^{(j)} = m^{(j)} + \log(\ell^{(j)})$。

在第 3.3 节的两块简单情形中，在线 softmax 技巧现在变为：

$$\begin{aligned}
m^{(1)} &= \mathrm{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \mathrm{rowsum}(e^{\mathbf{S}^{(1)}-m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{O}}^{(1)} &= e^{\mathbf{S}^{(1)}-m^{(1)}}\mathbf{V}^{(1)} \in \mathbb{R}^{B_r\times d} \\
m^{(2)} &= \max(m^{(1)}, \mathrm{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)}-m^{(2)}}\ell^{(1)} + \mathrm{rowsum}(e^{\mathbf{S}^{(2)}-m^{(2)}}) = \ell \\
\tilde{\mathbf{O}}^{(2)} &= \mathrm{diag}(e^{m^{(1)}-m^{(2)}})\tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)}-m^{(2)}}\mathbf{V}^{(2)} \\
\mathbf{O}^{(2)} &= \mathrm{diag}(\ell^{(2)})^{-1}\tilde{\mathbf{O}}^{(2)} = \mathbf{O}.
\end{aligned}$$

FlashAttention-2 前向传播算法改进详解

改进 1：延迟除以 ℓ
在 FlashAttention 中，每处理一块都要做：

$$\mathbf{O}^{(j)} = \underbrace{\mathrm{diag}(\ell^{(j-1)}/\ell^{(j)})}_{\text{rescale 旧输出}}\mathbf{O}^{(j-1)} + \underbrace{\mathrm{diag}(\ell^{(j)})^{-1}}_{\text{归一化新项}}e^{\mathbf{S}^{(j)}-m^{(j)}}\mathbf{V}^{(j)}$$

这里每次迭代都需要做两次除以 ℓ 的操作（对大小为 $B_r \times d$ 的矩阵），这是非 matmul 运算。
在 FlashAttention-2 中，改为维护未归一化的版本 $\tilde{\mathbf{O}}$：

$$\tilde{\mathbf{O}}^{(j)} = \mathrm{diag}(e^{m^{(j-1)}-m^{(j)}})\tilde{\mathbf{O}}^{(j-1)} + e^{\mathbf{S}^{(j)}-m^{(j)}}\mathbf{V}^{(j)}$$

只在最后一步才做一次除法：$\mathbf{O} = \mathrm{diag}(\ell^{(\mathrm{last})})^{-1}\tilde{\mathbf{O}}^{(\mathrm{last})}$。
节省了多少? 设有 T_c 个列块。FlashAttention 每次迭代做 2 次 rescale（每次涉及 $B_r \times d$ 个元素），共 $2T_c$ 次。FlashAttention-2 每次迭代只做 1 次 rescale（$\tilde{\mathbf{O}}$ 乘以 $e^{m^{(j-1)}-m^{(j)}}$），最后再做 1 次除法，共 $T_c + 1$ 次。减少了近一半的非 matmul 操作！
改进 2：只存 logsumexp
$L = m + \log(\ell)$
反向传播中需要的 softmax 概率可以通过 L 来恢复：

$$\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - L_i)$$

因为 $\exp(\mathbf{S}_{ij} - L_i) = \exp(\mathbf{S}_{ij} - m_i - \log \ell_i) = \frac{\exp(\mathbf{S}_{ij} - m_i)}{\ell_i}$
这正是 softmax 的定义！所以只存 L（一个 N 维向量）就够了，不需要分别存 m 和 ℓ（两个 N 维向量）。

注意事项

FlashAttention vs FlashAttention-2 的关键区别:
FlashAttention 在循环内维护**归一化后**的 $\mathbf{O}^{(j)}$，每次迭代需要 rescale 两项。
FlashAttention-2 在循环内维护**未归一化**的 $\tilde{\mathbf{O}}^{(j)}$，每次迭代只需 rescale 旧项（乘以 $e^{m^{(j-1)}-m^{(j)}}$），最后一步才做归一化。
这个看似微小的改变减少了约一半的非 matmul FLOPs，而这些非 matmul 运算在 GPU 上的代价是 matmul 的 16 倍！

我们展示完整的 FlashAttention-2 前向传播算法:

Algorithm 1 FlashAttention-2 前向传播

Require: 矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 在 HBM 中，块大小 B_c, B_r。
1: 将 $\mathbf{Q}$ 分为 $T_r = \lceil N/B_r \rceil$ 个大小为 $B_r \times d$ 的块 $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$；将 $\mathbf{K}, \mathbf{V}$ 分为 $T_c = \lceil N/B_c \rceil$ 个大小为 $B_c \times d$ 的块 $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ 和 $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$。
2: 将输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ 分为 T_r 个大小为 $B_r \times d$ 的块 $\mathbf{O}_1, \ldots, \mathbf{O}_{T_r}$；将 logsumexp L 分为 T_r 个大小为 B_r 的块 $L_1, \ldots, L_{T_r}$。
3: **for** $1 \le i \le T_r$ **do**
4: 　从 HBM 加载 $\mathbf{Q}_i$ 到片上 SRAM。
5: 　在片上初始化 $\mathbf{O}_i^{(0)} = (0)_{B_r \times d}$，$\ell_i^{(0)} = (0)_{B_r}$，$m_i^{(0)} = (-\infty)_{B_r}$。
6: 　**for** $1 \le j \le T_c$ **do**
7: 　　从 HBM 加载 $\mathbf{K}_j, \mathbf{V}_j$ 到片上 SRAM。
8: 　　在片上计算 $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$。
9: 　　在片上计算 $m_i^{(j)} = \max(m_i^{(j-1)}, \mathrm{rowmax}(\mathbf{S}_i^{(j)}))$，$\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)})$，$\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \mathrm{rowsum}(\tilde{\mathbf{P}}_i^{(j)})$。
10: 　　在片上计算 $\mathbf{O}_i^{(j)} = \mathrm{diag}(e^{m_i^{(j-1)} - m_i^{(j)}}) \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$。
11: 　**end for**
12: 　在片上计算 $\mathbf{O}_i = \mathrm{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$。
13: 　在片上计算 $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$。
14: 　将 $\mathbf{O}_i$ 写入 HBM 作为 $\mathbf{O}$ 的第 i 块。
15: 　将 L_i 写入 HBM 作为 L 的第 i 块。
16: **end for**
17: 返回输出 $\mathbf{O}$ 和 logsumexp L。

算法 1 逐行解读

第 1 行：将 $\mathbf{Q}$（$N \times d$）按行分块，每块 B_r 行；将 $\mathbf{K}, \mathbf{V}$ 按行分块，每块 B_c 行。
第 2 行：为输出和 logsumexp 分配空间。L 是一个 N 维向量，存储每行的 $\log(\sum e^{s_i})$。
第 3 行（外层循环）：遍历 $\mathbf{Q}$ 的每个行块。这个循环可以**并行化**（不同 i 之间完全独立）。
第 4 行：将当前 $\mathbf{Q}_i$（$B_r \times d$）从 HBM 加载到 SRAM。在整个内层循环中，$\mathbf{Q}_i$ 一直驻留在 SRAM 中。
第 5 行：初始化累加器。$\mathbf{O}_i^{(0)}$ 全零（未归一化的输出），$\ell_i^{(0)}$ 全零（指数和），$m_i^{(0)} = -\infty$（最大值设为负无穷以保证第一次比较时被覆盖）。

第 6–10 行（内层循环）：遍历 $\mathbf{K}, \mathbf{V}$ 的每个列块：

- 第 7 行：从 HBM 加载 $\mathbf{K}_j, \mathbf{V}_j$（各 $B_c \times d$）到 SRAM
- 第 8 行：**矩阵乘法**（matmul），计算局部注意力分数
- 第 9 行：更新统计量（**非 matmul**运算）——rowmax, exp, rowsum
- 第 10 行：更新输出——先 rescale 旧输出（非 matmul：逐元素乘），再加上新的 $\tilde{\mathbf{P}} \cdot \mathbf{V}$（**matmul**）

第 11 行：最后一步归一化：除以 ℓ。这是 FlashAttention-2 的关键改进——只做一次而非每次迭代做两次。
第 12 行：计算 logsumexp $L = m + \log(\ell)$，反向传播时使用。
第 13–14 行：将结果写回 HBM。每个行块只写一次，IO 量很小。

因果掩码 (Causal Masking).

原文翻译

注意力的一个常见用例是自回归语言建模，需要对注意力矩阵 $\mathbf{S}$ 应用因果掩码（即任何 $j > i$ 的项 $\mathbf{S}_{ij}$ 被设为 $-\infty$）。

1. 由于 FlashAttention 和 FlashAttention-2 已经按块操作，对于所有列索引都大于行索引的块（对于长序列约为一半的块），我们可以跳过该块的计算。这使得带因果掩码的注意力比不带掩码快约 1.7–1.8 倍。
2. 对于行索引保证严格小于列索引的块，我们不需要应用因果掩码。这意味着对于每一行，我们只需要对 1 个块（假设方块）应用因果掩码。

通俗解释

因果掩码让模型在生成第 i 个词时只能看到前 $i-1$ 个词（不能”偷看”未来）。这在注意力矩阵中形成一个**下三角**：

$$\mathbf{S}_{\text{masked}} = \begin{pmatrix} s_{11} & -\infty & -\infty & -\infty \\ s_{21} & s_{22} & -\infty & -\infty \\ s_{31} & s_{32} & s_{33} & -\infty \\ s_{41} & s_{42} & s_{43} & s_{44} \end{pmatrix}$$

由于分块计算，很多块完全在对角线之上（全是 $-\infty$，softmax 后全是 0），可以直接跳过。只有对角线附近的块需要实际应用掩码，约节省一半计算量。

正确性、运行时间和内存需求。

原文翻译

与 FlashAttention 一样，算法 1 返回正确的输出 $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$（无近似），使用 $O(N^2 d)$ FLOPs，且在输入和输出之外只需要 $O(N)$ 额外内存（用于存储 logsumexp L）。证明与 Dao et al. [2022] 的定理 1 几乎相同，此处省略。

4.1.2 反向传播

原文翻译

FlashAttention-2 的反向传播与 FlashAttention 几乎相同。我们做了一个小调整：只使用按行的 logsumexp L，而非同时使用按行的最大值和指数和。我们在算法 2 中给出完整的反向传播描述。

Algorithm 2 FlashAttention-2 反向传播

Require: 矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ 在 HBM 中，向量 $L \in \mathbb{R}^N$ 在 HBM 中，块大小 B_c，B_r。

1: 将 $\mathbf{Q}$ 分为 $T_r = \lceil N/B_r \rceil$ 块，将 $\mathbf{K}, \mathbf{V}$ 分为 $T_c = \lceil N/B_c \rceil$ 块。
2: 将 $\mathbf{O}$ 分为 T_r 块，$\mathbf{dO}$ 分为 T_r 块，L 分为 T_r 块。
3: 初始化 $\mathbf{dQ} = (0)_{N \times d}$ 在 HBM 中并分为 T_r 块。划分 $\mathbf{dK}, \mathbf{dV}$ 为 T_c 块。
4: 计算 $D = \mathrm{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^N$（逐元素乘法），写入 HBM 并分为 T_r 块。
5: **for** $1 \le j \le T_c$ **do**
6: 　从 HBM 加载 $\mathbf{K}_j, \mathbf{V}_j$ 到片上 SRAM。
7: 　在 SRAM 上初始化 $\mathbf{dK}_j = (0)_{B_c \times d}$，$\mathbf{dV}_j = (0)_{B_c \times d}$。
8: 　**for** $1 \le i \le T_r$ **do**
9: 　　从 HBM 加载 $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ 到片上 SRAM。
10: 　　在片上计算 $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$。
11: 　　在片上计算 $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$。
12: 　　在片上计算 $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$。
13: 　　在片上计算 $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$。
14: 　　在片上计算 $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$。
15: 　　从 HBM 加载 $\mathbf{dQ}_i$ 到 SRAM，在片上更新 $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j$，写回 HBM。
16: 　　在片上计算 $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$。
17: 　**end for**
18: 　将 $\mathbf{dK}_j, \mathbf{dV}_j$ 写入 HBM。
19: **end for**
20: 返回 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$。

算法 2 逐行解读

反向传播的目标是计算损失对 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 的梯度 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$。

第 4 行：预先计算 $D = \mathrm{rowsum}(\mathbf{dO} \circ \mathbf{O})$。这是 softmax 梯度公式中需要的“校正项”：$\mathbf{dS} = \mathbf{P} \circ (\mathbf{dP} - D)$。$D_i$ 就是第 i 行的 $\sum_k p_{ik} \cdot dp_{ik}$。

第 5–18 行（外层循环）：遍历 $\mathbf{K}, \mathbf{V}$ 的每个列块。注意**外层循环是列块**，与前向传播（外层是行块）不同！这是因为 $\mathbf{dK}_j$ 和 $\mathbf{dV}_j$ 需要在内层循环中累加来自所有行块的贡献。

内层循环核心步骤：

- 第 10 行：**重计算 S**（matmul）
- 第 11 行：用 logsumexp L **重计算 P**。关键公式：$\mathbf{P} = \exp(\mathbf{S} - L)$，无需分别存 m 和 ℓ
- 第 12 行：累加 $\mathbf{dV}$（matmul）：$\mathbf{dV}_j += \mathbf{P}^\top \mathbf{dO}_i$
- 第 13 行：计算 $\mathbf{dP}$（matmul）：$\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top$

- 第 14 行：计算 $\mathbf{dS}$（非 matmul）：$\mathbf{dS} = \mathbf{P} \circ (\mathbf{dP} - D)$
- 第 15 行：更新 $\mathbf{dQ}$（matmul + HBM 读写）：$\mathbf{dQ}_i += \mathbf{dS}\mathbf{K}_j$
- 第 16 行：累加 $\mathbf{dK}$（matmul）：$\mathbf{dK}_j += \mathbf{dS}^\top \mathbf{Q}_i$

反向传播共有 5 次 matmul，比前向传播（2 次）多 2.5 倍，这也是反向传播 FLOPs 为前向的 2.5 倍的原因。

Multi-query Attention 和 Grouped-query Attention.

原文翻译

Multi-query attention (MQA) [Shazeer, 2019]【Fast Transformer Decoding：提出多 query 头共享一个 KV 头，减少推理时 KV 缓存大小】和 grouped-query attention (GQA) [Ainslie et al., 2023]【GQA：多个 query 头共享一组 KV 头，是 MQA 和 MHA 的折中方案】是注意力的变体，其中多个 query 头对应同一个 key 和 value 头，以减少推理时 KV 缓存的大小。我们不需要复制 key 和 value 头来进行计算，而是隐式操作头的索引来执行相同的计算。在反向传播中，我们需要对被隐式复制的不同头的梯度 $\mathbf{dK}$ 和 $\mathbf{dV}$ 求和。

4.2 并行性 (Parallelism)

原文翻译

FlashAttention 的第一个版本在 batch size 和注意力头数上进行并行化。我们使用 1 个线程块处理 1 个注意力头，总共有 batch size × 头数 个线程块。每个线程块被调度到一个流式多处理器（SM）上运行，以 A100 GPU 为例有 108 个 SM。当这个数字较大（比如 ≥ 80）时，这种调度是高效的，因为我们可以有效利用 GPU 上几乎所有的计算资源。
在长序列的情况下（通常意味着小 batch size 或少量头），为了更好地利用 GPU 上的多处理器，我们现在额外在序列长度维度上进行并行化。这在这种场景下带来了显著加速。

前向传播.

原文翻译

我们看到外层循环（遍历序列长度）是**尴尬并行**（embarrassingly parallel）的，我们将它们调度到不同的线程块上，这些线程块之间不需要互相通信。我们还像 FlashAttention 一样在 batch 维度和头数维度上并行化。序列长度上增加的并行性有助于在 batch size 和头数较小时提高占用率（GPU 资源使用比例），从而在这种情况下带来加速。
交换循环顺序的想法（外层循环遍历行块、内层循环遍历列块，而非原始 FlashAttention 论文中的相反顺序），以及在序列长度维度上的并行化，最先由 Phil Tillet 在 Triton [Tillet et al., 2019]【Triton：一种用于编写高效 GPU 核函数的中间语言和编译器】实现中提出并实现。

反向传播.

原文翻译

注意不同列块之间唯一共享的计算是更新 $\mathbf{dQ}$（算法 2 中），需要从 HBM 加载 $\mathbf{dQ}_i$ 到 SRAM，在片上更新 $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)}\mathbf{K}_j$，然后写回 HBM。因此我们也在序列长度维度上并行化，并为反向传播的每个列块分配 1 个线程块。我们使用原子加操作（atomic adds）在不同线程块之间通信以更新 $\mathbf{dQ}$。

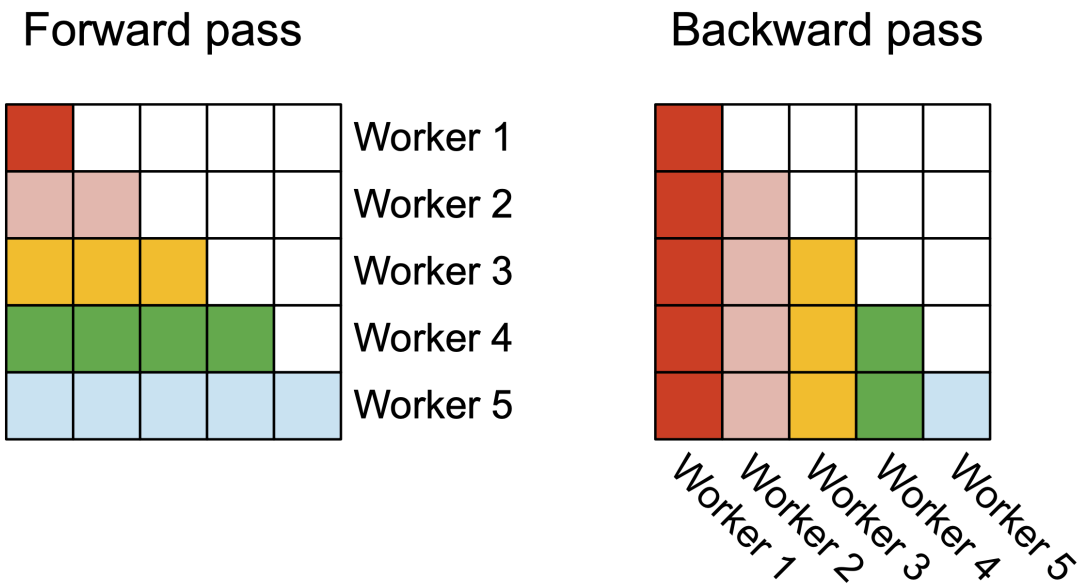


Figure 2: 前向传播（左）中，我们将工作线程（线程块）并行化，每个工作线程负责注意力矩阵的一个行块。反向传播（右）中，每个工作线程负责注意力矩阵的一个列块。

图 2 解读

左图（前向传播）：

- 注意力矩阵 $\mathbf{S}$ 的每一行块由一个独立的线程块处理
- 行块之间完全独立，不需要通信——因为每行的 softmax 是独立的
- 这就是”尴尬并行”：分工简单且完美

右图（反向传播）：

- 每一列块由一个独立的线程块处理
- $\mathbf{dK}_j$ 和 $\mathbf{dV}_j$ 在列块内完全累加，不需要跨线程块通信
- 唯一的例外是 $\mathbf{dQ}_i$，因为不同列块都需要往同一个 $\mathbf{dQ}_i$ 里累加，需要使用**原子加操作**

为什么前向按行、反向按列?
前向传播中，同一行的 softmax 归一化需要完整的行信息，所以一个线程块处理完整行。
反向传播中，$\mathbf{dK}_j$ 和 $\mathbf{dV}_j$ 需要从**所有行块**收集贡献到同一个 K/V 块，所以一个线程块处理完整列更高效。

4.3 Warp 间的工作分配 (Work Partitioning Between Warps)

原文翻译

正如第 4.2 节描述了线程块的调度，即使在每个线程块内部，我们也需要决定如何在不同 warp 之间分配工作。我们通常使用每个线程块 4 或 8 个 warp，分配方式描述在图 3 中。

前向传播.

原文翻译

对于每个块，FlashAttention 将 **K** 和 **V** 在 4 个 warp 间拆分，同时让 **Q** 对所有 warp 可访问。每个 warp 乘以得到 $\mathbf{QK}^\top$ 的一个切片，然后它们需要与 **V** 的一个切片相乘并通信以累加结果。这被称为“split-K”方案。然而，这是低效的，因为所有 warp 需要将它们的中间结果写出到共享内存、同步、然后累加中间结果。这些共享内存读写拖慢了 FlashAttention 的前向传播。

在 FlashAttention-2 中，我们改为将 **Q** 在 4 个 warp 间拆分，同时让 **K** 和 **V** 对所有 warp 可访问。每个 warp 执行矩阵乘法得到 $\mathbf{QK}^\top$ 的一个切片后，它们只需要与共享的 **V** 切片相乘即可得到其对应的输出切片。warp 之间不需要通信。共享内存读写的减少带来了加速（第 5 节）。

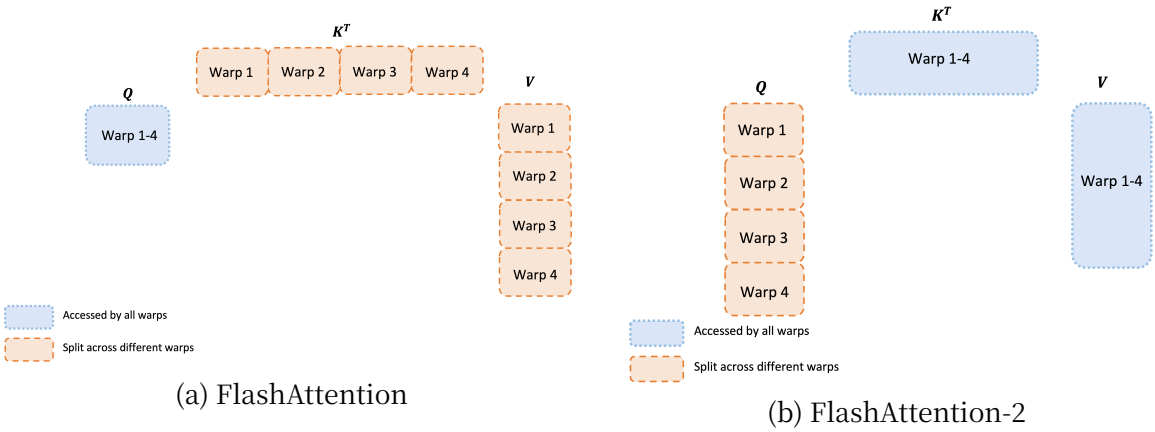


Figure 3: 前向传播中不同 warp 之间的工作分配

图 3 解读：warp 分工对比

图 (a) FlashAttention 的 split-K 方案:

- 4 个 warp 各自负责 **K** 的一个切片
- 每个 warp 计算 $\mathbf{Q} \cdot (\mathbf{K}_{\text{slice}})^\top$ 得到 **S** 的一列切片
- 然后各 warp 用自己的 **V** 切片算出输出的一部分
- **问题**：最后需要所有 warp 的结果**相加**，这需要通过共享内存通信
- 通信流程：warp→共享内存→同步→读取累加，这很慢

图 (b) FlashAttention-2 的 split-Q 方案:

- 4 个 warp 各自负责 **Q** 的一个切片（即输出行的不同子集）

- 每个 warp 计算自己的 $\mathbf{Q}_{\text{slice}} \cdot \mathbf{K}^\top$ 得到 $\mathbf{S}$ 的一行切片
- 然后用**完整的 V**（所有 warp 共享读取）算出自己的输出行
- **优势**：每个 warp 的输出行是独立的，不需要跨 warp 累加，零通信！

类比：想象 4 个厨师（warp）要做一桌菜：

- FlashAttention 方式：每个厨师负责切一种配料，然后大家把切好的配料堆到一个盘子里炒——需要排队等共用灶台
- FlashAttention-2 方式：每个厨师负责做一道完整的菜（用共享的配料），各做各的互不干扰

反向传播.

原文翻译

类似地，在反向传播中，我们选择分配 warp 以避免“split-K”方案。然而，由于 $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO}, \mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ 之间更复杂的依赖关系，仍需要一些同步。尽管如此，避免“split-K”减少了共享内存读写并再次带来了加速（第 5 节）。

块大小调优.

原文翻译

增大块大小通常能减少共享内存加载/存储，但会增加所需的寄存器数量和共享内存总量。超过某个块大小后，寄存器溢出（register spilling）会导致显著减速，或者所需的共享内存量超过 GPU 可用量，kernel 根本无法运行。通常我们选择 $\{64, 128\} \times \{64, 128\}$ 大小的块，取决于头维度 d 和设备共享内存大小。
我们为每个头维度手动调优，因为本质上只有 4 种块大小选择，但这可以通过自动调优来避免手动工作。我们将此留作未来工作。

关键概念

FlashAttention-2 的三大改进总结：

1. **算法层面**：延迟除以 ℓ 到循环结束，只存 logsumexp L 而非分别存 m 和 ℓ，减少约一半非 matmul FLOPs
2. **线程块并行**：在序列长度维度上增加并行化，提高长序列场景下的 GPU 占用率
3. **Warp 分工**：从 split-K（拆 **K**）改为 split-Q（拆 **Q**），消除 warp 间的共享内存通信

5 实验验证 (Empirical Validation)

原文翻译

我们评估使用 FlashAttention-2 训练 Transformer 模型的效果。

- **注意力基准测试。** 我们在不同序列长度下测量 FlashAttention-2 的运行时间，并与 PyTorch 标准实现、FlashAttention 和 Triton 中的 FlashAttention 进行比较。我们确认 FlashAttention-2 比 FlashAttention 快 1.7–3.0 倍，比 Triton 中的 FlashAttention 快 1.3–2.5 倍，比标准注意力实现快 3–10 倍。FlashAttention-2 在 A100 GPU 上达到高达 230 TFLOPs/s，即理论最大 TFLOPs/s 的 73%。
- **端到端训练速度。** 当端到端用于训练参数量为 1.3B 和 2.7B、序列长度为 2k 或 8k 的 GPT 风格模型时，FlashAttention-2 相比 FlashAttention 带来高达 1.3 倍的加速，相比不使用 FlashAttention 的基线带来 2.8 倍的加速。FlashAttention-2 在每个 A100 GPU 上达到高达 225 TFLOPs/s（72% 模型 FLOPs 利用率）。

5.1 注意力基准测试 (Benchmarking Attention)

原文翻译

我们在 A100 80GB SXM4 GPU 上测量不同注意力方法在不同设置（有/无因果掩码，头维度 64 或 128）下的运行时间。
基准测试设置：我们将序列长度从 512, 1k, ..., 16k 变化，并设置 batch size 使得总 token 数为 16k。隐藏维度设为 2048，头维度为 64 或 128（即 32 个头或 16 个头）。
前向传播 FLOPs 的计算公式：

$$4 \times \text{seqlen}^2 \times \text{head_dim} \times \text{num_heads}. \tag{7}$$

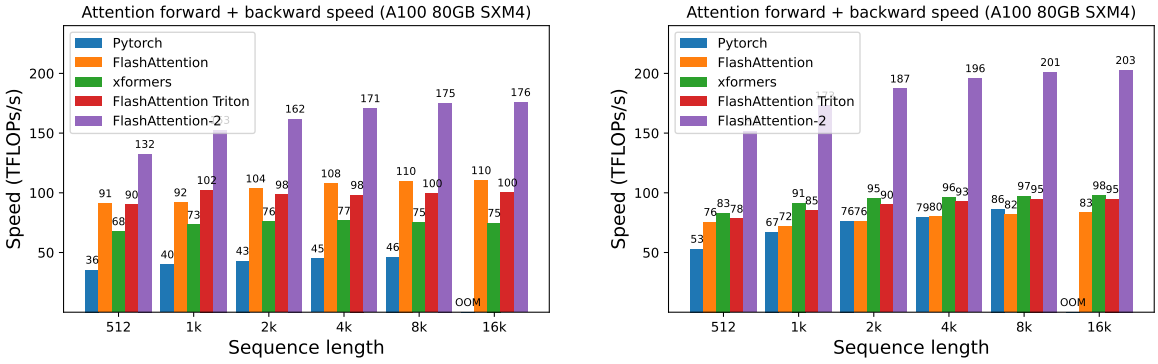
使用因果掩码时，将此数除以 2（因为约只计算一半的元素）。反向传播 FLOPs 为前向的 2.5 倍（因为前向有 2 次 matmul，反向有 5 次 matmul，含重计算）。

FLOPs 计算公式推导

前向传播 FLOPs
注意力前向包含 2 次主要矩阵乘法：

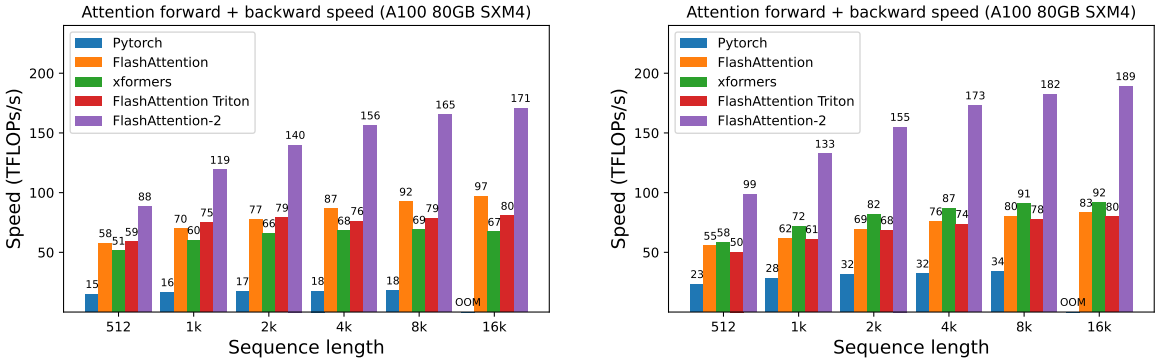
1. $\mathbf{S} = \mathbf{QK}^\top$：$\mathbf{Q}$ 是 $(N \times d)$，$\mathbf{K}^\top$ 是 $(d \times N)$，结果 $(N \times N)$。FLOPs $= 2Nd \cdot N = 2N^2d$（每个输出元素需 d 次乘法和 $d-1$ 次加法 $\approx 2d$ FLOPs）。
2. $\mathbf{O} = \mathbf{PV}$：$\mathbf{P}$ 是 $(N \times N)$，$\mathbf{V}$ 是 $(N \times d)$，结果 $(N \times d)$。FLOPs $= 2N^2d$。

总计 $= 2 \times 2N^2d = 4N^2d$（per head）。
对于多头，总 FLOPs $= 4 \times N^2 \times d \times h$，其中 h 是头数。
数值例子：$N = 2048$，$d = 128$，$h = 16$：
$4 \times 2048^2 \times 128 \times 16 = 4 \times 4.19 \times 10^6 \times 128 \times 16 \approx 34.4 \times 10^9 = 34.4$ GFLOPs
反向传播 FLOPs
反向有 5 次 matmul（$\mathbf{P}^\top\mathbf{dO}$, $\mathbf{dOV}^\top$, $\mathbf{dSK}$, $\mathbf{dS}^\top\mathbf{Q}$, 加上重计算 $\mathbf{QK}^\top$），所以是前向的 $5/2 = 2.5$ 倍。



(a) 无因果掩码, 头维度 64

(b) 无因果掩码, 头维度 128



(c) 有因果掩码, 头维度 64

(d) 有因果掩码, 头维度 128

Figure 4: A100 GPU 上注意力前向+反向速度

图 4 解读

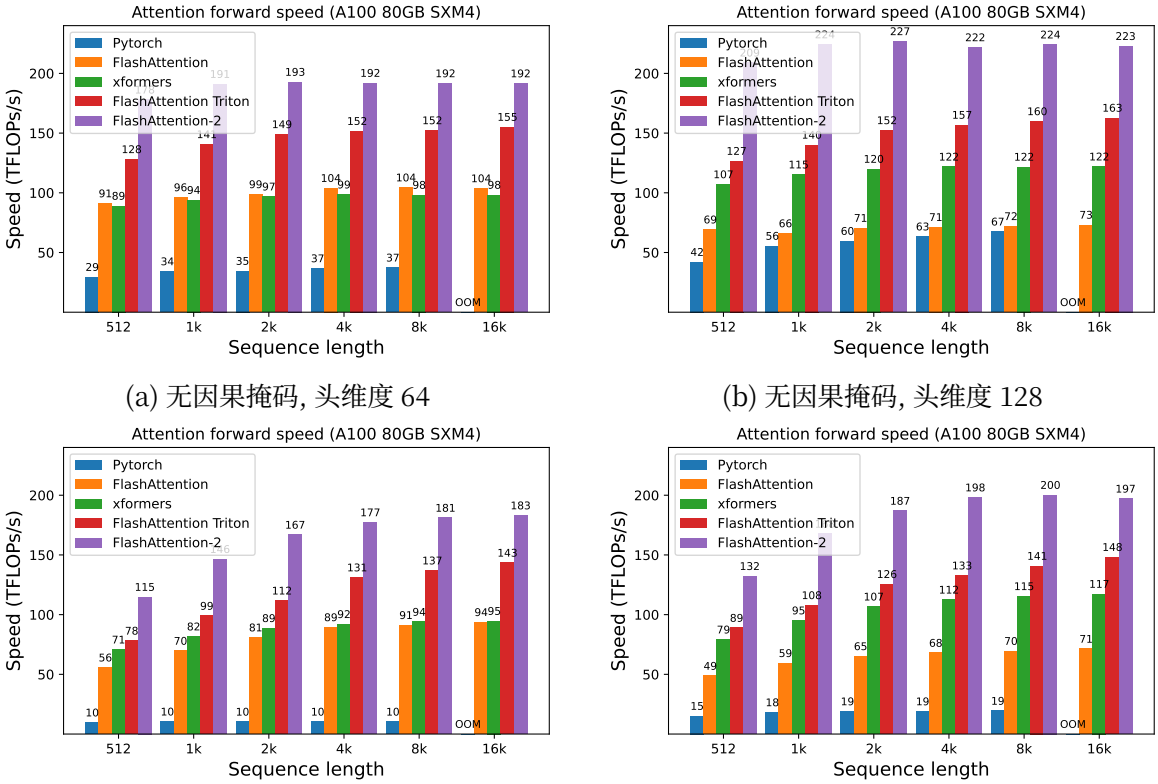
这组图展示了 A100 GPU 上前向+反向总速度（TFLOPs/s），横轴是序列长度，纵轴是吞吐量。

各条线的含义:

- **FlashAttention-2**（最上方）：在所有设置下都最快
- **FlashAttention in Triton**：Phil Tillet 的 Triton 实现，次之
- **FlashAttention**：原始 CUDA 实现
- **PyTorch standard**（最下方）：标准注意力实现，最慢

关键观察:

- FlashAttention-2 比 FlashAttention 快约 2 倍（前向+反向综合）
- 序列越长，FlashAttention-2 的优势越明显（GPU 占用率更高）
- 头维度 128 比 64 的吞吐量更高（更大的矩阵，Tensor Core 效率更高）
- 有因果掩码时吞吐量更高（跳过约一半的块）



(a) 无因果掩码, 头维度 64

(b) 无因果掩码, 头维度 128

(c) 有因果掩码, 头维度 64

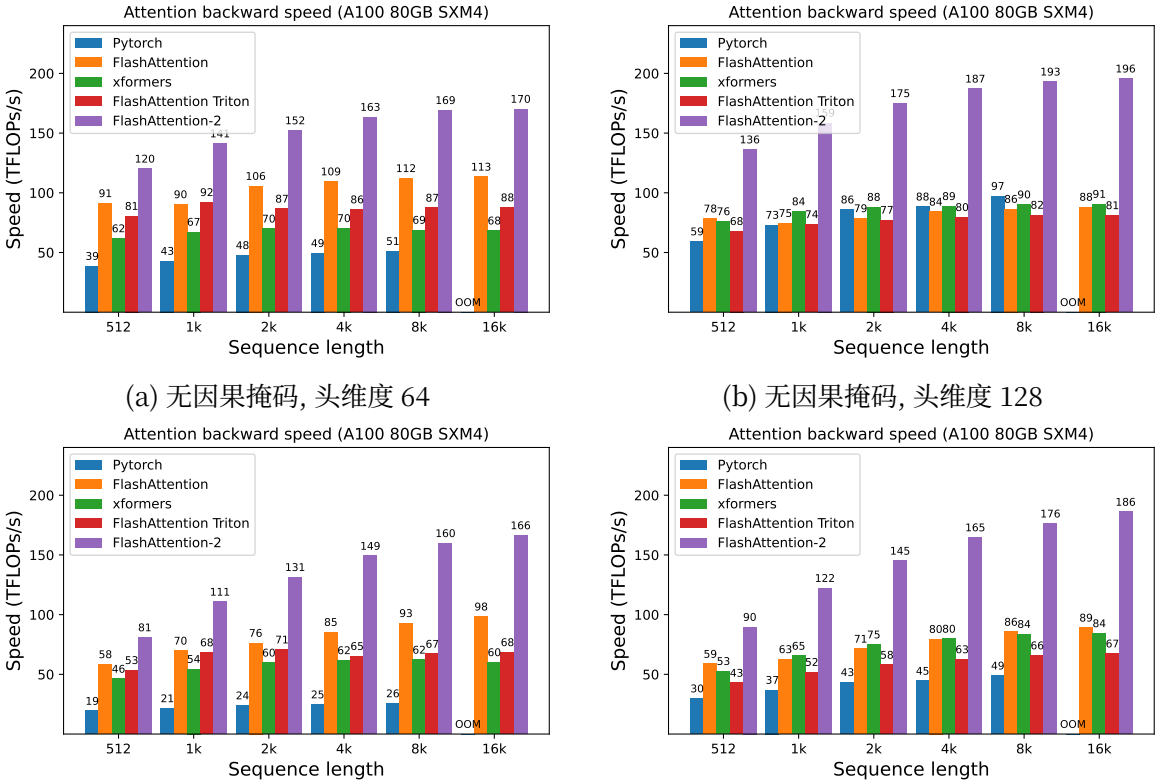
(d) 有因果掩码, 头维度 128

Figure 5: A100 GPU 上注意力前向速度

图 5 解读

仅前向传播的性能:

- FlashAttention-2 在前向传播中达到理论最大吞吐量的 **73%**，非常接近 GEMM 的效率
- 比 Triton 中的 FlashAttention 快 1.3–1.5 倍
- 比 FlashAttention 原始实现快约 2 倍
- 序列长度越长，效率越高（更好地摊销了 kernel launch 等固定开销）



(a) 无因果掩码, 头维度 64

(b) 无因果掩码, 头维度 128

(c) 有因果掩码, 头维度 64

(d) 有因果掩码, 头维度 128

Figure 6: A100 GPU 上注意力反向速度

图 6 解读

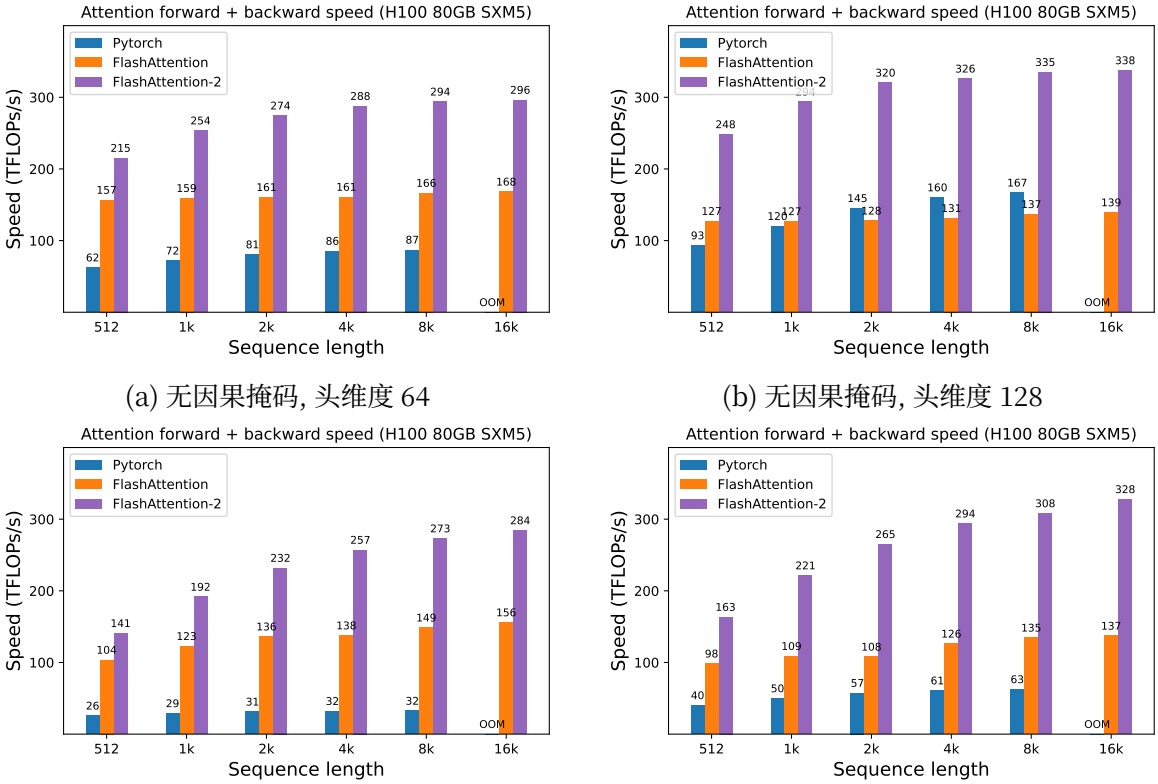
仅反向传播的性能。反向传播达到理论最大吞吐量的 **63%**，低于前向的 73%，原因是：

- 反向传播需要 5 次 matmul + 更多非 matmul 运算
- 需要更多 SRAM 存储中间值（$\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO}$ 等）
- $\mathbf{dQ}$ 的更新需要 HBM 往返（或原子加操作）

FlashAttention-2 比 FlashAttention 快约 2 倍，比 Triton 版快约 2 倍。

原文翻译

在 H100 GPU 上运行相同的实现（未使用 TMA 和第 4 代 Tensor Core 等新特性），我们获得了高达 335 TFLOPs/s（图 7）。我们预期通过使用新指令，可以在 H100 GPU 上获得额外 1.5–2 倍的加速。我们将此留作未来工作。



(a) 无因果掩码, 头维度 64

(b) 无因果掩码, 头维度 128

(c) 有因果掩码, 头维度 64

(d) 有因果掩码, 头维度 128

Figure 7: H100 GPU 上注意力前向+反向速度

图 7 解读

这组图展示了在更新一代的 H100 GPU 上的结果。虽然没有使用 H100 的新特性（如 TMA 即 Tensor Memory Accelerator、第 4 代 Tensor Core、FP8），仅直接运行 A100 版本的代码，已经达到 335 TFLOPs/s。H100 的理论峰值远高于 A100（FP16 约 989 TFLOPs/s），说明还有很大的优化空间。

5.2 端到端性能 (End-to-end Performance)

原文翻译

我们测量 GPT 风格模型（1.3B 和 2.7B 参数）在 8× A100 80GB SXM 上的训练吞吐量。如表 1 所示，FlashAttention-2 相比不使用 FlashAttention 的基线带来 2.8 倍加速，相比 FlashAttention 带来 1.3 倍加速，在每个 A100 GPU 上达到高达 225 TFLOPs/s。注意我们使用以下公式计算 FLOPs，遵循 Megatron-LM [Shoeybi et al., 2019]【Megatron-LM：大规模分布式 Transformer 训练框架，提出了模型并行训练的方法】：

$$6 \times \text{seqlen} \times \text{params} + 12 \times \text{layers} \times \text{hidden_dim} \times \text{seqlen}^2. \tag{8}$$

第一项是权重–输入乘法的 FLOPs，第二项是注意力的 FLOPs。虽然有人认为第二项应减半（因果掩码只计算约一半元素），我们选择遵循文献中的公式（不除以 2）以保持一致性。

Table 1: GPT 风格模型在 8× A100 GPU 上的训练速度（TFLOPs/s/GPU）。FlashAttention-2 达到高达 225 TFLOPs/s（72% 模型 FLOPs 利用率）。与不使用 FlashAttention 的基线进行比较。

模型	无 FlashAttention	FlashAttention	FlashAttention-2
GPT3-1.3B 2k context	142 TFLOPs/s	189 TFLOPs/s	196 TFLOPs/s
GPT3-1.3B 8k context	72 TFLOPs/s	170 TFLOPs/s	220 TFLOPs/s
GPT3-2.7B 2k context	149 TFLOPs/s	189 TFLOPs/s	205 TFLOPs/s
GPT3-2.7B 8k context	80 TFLOPs/s	175 TFLOPs/s	**225 TFLOPs/s**

表 1 解读

表格展示的对比关系：三种实现在不同模型大小和上下文长度下的端到端训练吞吐量。
关键发现：

- **长序列收益最大**：在 8k 上下文时，FlashAttention-2 比基线快 $225/80 = 2.8\times$，比 FlashAttention 快 $225/175 = 1.29\times$。在 2k 上下文时收益较小（$196/142 = 1.38\times$ vs 基线）。
- **原因**：长序列意味着注意力占总计算量的比例更大（注意力是 $O(N^2)$，FFN 是 $O(N)$）。FlashAttention 只加速注意力部分，因此序列越长，注意力计算占比越高，整体收益越大。
- **225 TFLOPs/s 意味着什么？** A100 的 FP16 理论峰值是 312 TFLOPs/s。$225/312 = 72\%$ 的模型 FLOPs 利用率（MFU），这是非常高的。作为参考，很多大模型训练的 MFU 只有 30–50%。

端到端 FLOPs 公式推导

公式 (8)：$6 \cdot N \cdot P + 12 \cdot L \cdot H \cdot N^2$
第一项： $6 \cdot N \cdot P$
Transformer 中每个 token 经过权重矩阵的计算量。因子 6 = 前向 2（乘法+加法）× 3（前向+梯度计算+权重更新各一次）。P 是参数量，N 是序列长度。
第二项： $12 \cdot L \cdot H \cdot N^2$
注意力计算。每层有 $4N^2d$ 前向 FLOPs（$d = H/L$ 每头维度 × 头数时简化），因子 12 包含前向+反向倍数。
数值例子：GPT3-2.7B, seqlen=8k
设 $P = 2.7 \times 10^9$, $N = 8192$, $L = 32$, $H = 2560$：

- 权重项：$6 \times 8192 \times 2.7 \times 10^9 \approx 1.33 \times 10^{14}$
- 注意力项：$12 \times 32 \times 2560 \times 8192^2 \approx 6.60 \times 10^{13}$
- 总计 $\approx 1.99 \times 10^{14} \approx 200$ TFLOPs per step

若在 8 块 A100 上每秒完成 ~9 步，则每 GPU 吞吐量 $\approx 200 \times 9/8 = 225$ TFLOPs/s。

6 讨论与未来方向 (Discussion and Future Directions)

原文翻译

FlashAttention-2 比 FlashAttention 快 2 倍，这意味着我们可以用之前训练 8k 上下文模型的相同成本训练 16k 更长上下文的模型。我们对这如何用于理解长书籍和报告、高分辨率图像、音频和视频感到兴奋。FlashAttention-2 还将加速现有模型的训练、微调和推理。

在近期，我们计划与研究人员和工程师合作，使 FlashAttention 广泛适用于不同类型的设备（如 H100 GPU、AMD GPU），以及新的数据类型如 FP8。作为直接的下一步，我们计划优化 FlashAttention-2 以利用 H100 GPU 的新硬件特性（TMA、第 4 代 Tensor Core、FP8）。将 FlashAttention-2 中的底层优化与高层算法变化（如局部、膨胀、块稀疏注意力）相结合，可以让我们用更长的上下文训练 AI 模型。我们也期待与编译器研究人员合作，使这些优化技术更容易编程。

通俗解释

讨论部分展望了四个方向：

1. **更多硬件支持**：H100、AMD GPU 等
2. **新数据类型**：FP8 精度（更低精度意味着更高吞吐量）
3. **结合稀疏注意力**：FlashAttention 处理的是精确注意力，如果与局部/稀疏注意力模式结合，可以进一步扩展上下文长度
4. **编译器友好**：让这些优化更容易被自动化工具使用

致谢 (Acknowledgments)

原文翻译

我们感谢 Phil Tillet 和 Daniel Haziza，他们分别在 Triton [Tillet et al., 2019] 和 xformers 库 [Lefaudeux et al., 2022] 【xformers：Facebook 开源的 Transformer 模块化库，包含多种高效注意力实现】中实现了 FlashAttention 的版本。FlashAttention-2 的灵感来自于不同注意力实现方式之间的思想交流。

我们感谢 Nvidia CUTLASS 团队（特别是 Vijay Thakkar、Cris Cecka、Haicheng Wu 和 Andrew Kerr）的 CUTLASS 库，特别是 CUTLASS 3.x 版本，它提供了清晰的抽象和强大的构建块，用于 FlashAttention-2 的实现。

我们感谢 Driss Guessous 将 FlashAttention 集成到 PyTorch 中。FlashAttention-2 受益于与 Phil Wang、Markus Rabe、James Bradbury、Young-Jun Ko、Julien Launay、Daniel Hesslow、Michaël Benesty、Horace He、Ashish Vaswani 和 Erich Elsen 的有益讨论。

感谢 Stanford CRFM 和 Stanford NLP 的计算支持。感谢 Dan Fu 和 Christopher Ré 的合作、建设性反馈和持续鼓励。感谢 Albert Gu 和 Beidi Chen 对本技术报告早期草稿的有益建议。

附录：参考文献一览表

文献	主要内容	在本文中被引用的原因
Vaswani et al., 2017 (Attention is All You Need)	提出 Transformer 架构和自注意力机制	本文优化的核心对象——注意力机制的来源
Dao et al., 2022 (FlashAttention)	提出利用 GPU 内存层次加速精确注意力的 IO 感知算法	FlashAttention-2 的直接前作，本文在其基础上改进
OpenAI, 2023 (GPT-4)	GPT-4 技术报告，32k 上下文	说明长上下文模型的需求和趋势
Milakov & Gimelshein, 2018 (Online Softmax)	提出在线计算 softmax 归一化因子的方法	FlashAttention 的在线 softmax 核心技术来源
Rabe & Staats, 2021	证明自注意力可以 $O(n)$ 内存实现	在线 softmax 用于注意力的理论依据
Kitaev et al., 2020 (Reformer)	局部敏感哈希注意力近似	作为近似注意力方法的代表被引用
Roy et al., 2021 (Routing Transformers)	路由机制的稀疏注意力	同上
Wang et al., 2020 (Linformer)	低秩近似实现线性复杂度注意力	同上
Katharopoulos et al., 2020 (Linear Attention)	将 Transformer 视为 RNN 的线性注意力	同上
Choromanski et al., 2020 (Performer)	随机特征近似 softmax 核	同上
Beltagy et al., 2020 (Longformer)	局部滑动窗口+全局注意力	同上
Zaheer et al., 2020 (BigBird)	局部+全局+随机注意力	同上
Chen et al., 2021 (ScatterBrain)	统一稀疏和低秩注意力	同上
Shazeer, 2019 (MQA)	Multi-Query Attention，多 head 共享 KV	本文支持 MQA/GQA 的高效实现
Ainslie et al., 2023 (GQA)	Grouped-Query Attention	同上，MQA 和 MHA 的折中
Tillet et al., 2019 (Triton)	GPU kernel 编译器	FlashAttention 的 Triton 实现作为基准对比
Lefaudeux et al., 2022 (xformers)	模块化 Transformer 库	FlashAttention 的 xformers 实现作为基准对比

文献	主要内容	在本文中被引用的原因
Jia et al., 2018	Volta GPU 微基准测试	提供 GPU 内存带宽参数
Jia & Van Sandt, 2021	Ampere GPU 微基准测试	提供 A100 性能参数
Shoeybi et al., 2019 (Megatron-LM)	大规模分布式 Transformer 训练	FLOPs 计算公式来源